Machine Learning Operations

Lecture 5: Foundation model fine-tuning

Daniel Dahlmeier, Christopher Teo

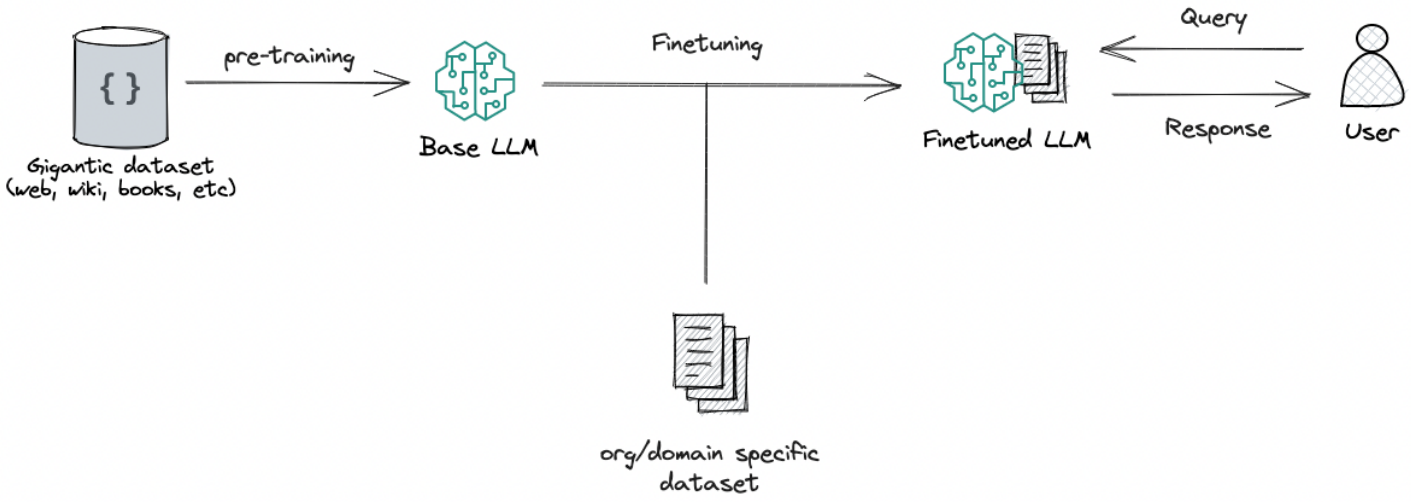
November 6, 2025

Agenda

- Foundation model fine-tuning
- Parameter-efficient fine-tuning
- Instruction tuning
- GenAI architecture patterns

Foundation model fine-tuning

Finetuning



What is finetuning?

- Further train a base LLM to adapt it for a specific task or enhance performance
- Turn general model into specialized model, e.g., GPT-4 into Github Copilot
- Actual *learning*, i.e., update of weight parameters, instead of just access to more data

https://towardsdatascience.com/rag-vs-finetuning-which-is-the-best-tool-to-boost-your-llm-application-94654b1eaba7

Finetuning

Why finetune?

- Adopt to new behaviors, can gain new knowledge
- Steers the model to more consistent outputs or be aligned with human values
- Reduce hallucinations
- Customize model to specific use case

Finetuning can mean different things

Types of finetuning:

- Continued LLM pre-training with domain-specific data
- Instruction tuning with instruction-output pairs to follow instructions
- Single-task fine-tuning to improve performance on a specific task such as toxicity detection or summarization
- Reinforcement learning with human feedback (RLHF) combines instruction fine-tuning with reinforcement learning

https://eugeneyan.com/writing/llm-patterns/

Finetuning and RAG address different issues

	RAG	Finetuning
External knowledge req'd?	✓	✗
Changing model behaviour req'd?	✗	✓
Minimise hallucinations?	✓	✗
Training data available?	✗	✓
Is data (mostly) dynamic?	✓	✗
Interpretability req'd?	✓	✗

https://www.linkedin.com/pulse/rag-vs-finetuning-which-best-your-llm-applications-sarfraz-nawaz/

Finetuning allows to adopt model behavior, RAG grounds the model in data

Fine-tuning vs RAG

Fine-Tuning (or Retraining)

- Need to train the LLM to perform a different task e.g., generate code
- Expensive to collect training data and train the model
- Pro data science skills to optimize the objective function

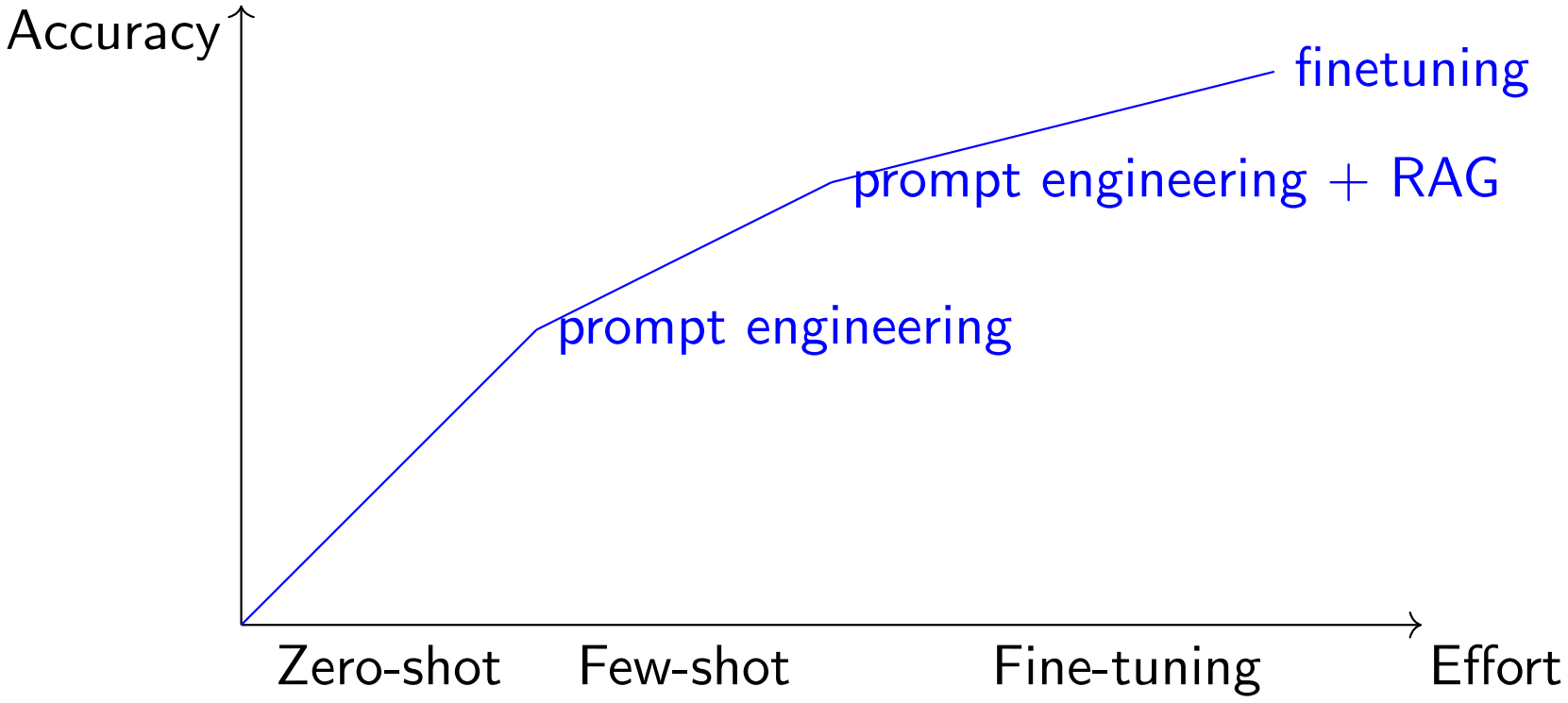
Retrieval Augmented Generation (RAG)

- Adapt the model to a new domain
- Constrain the model to existing knowledge base
- Cost-efficient as compared to fine-tuning
- Low-code / no-code skills

- Fine-tuned smaller models can be more cost efficient in production than large prompted models

https://www.linkedin.com/pulse/rag-vs-finetuning-which-best-your-llm-applications-sarfraz-nawaz/

Finetuning



- Start with zero-shot prompt engineering
- Then augment prompt with (retrieved) context, e.g., few-shot learning
- Then invest into fine-tuning OSS models, start with small models
- Only if you have a good reason: build your own LLM from scratch

Finetuning strategies

Training strategy

- Full finetuning
- PEFT (e.g., BitFit, LoRA, QLoRA)

Model family

- Pythia
- Llama
- Mistral
- Gemma
- Qwen

Model type

- Base model
- Chat model
- Reasoning Model

Model size

- 1B
- 7B
- 70B

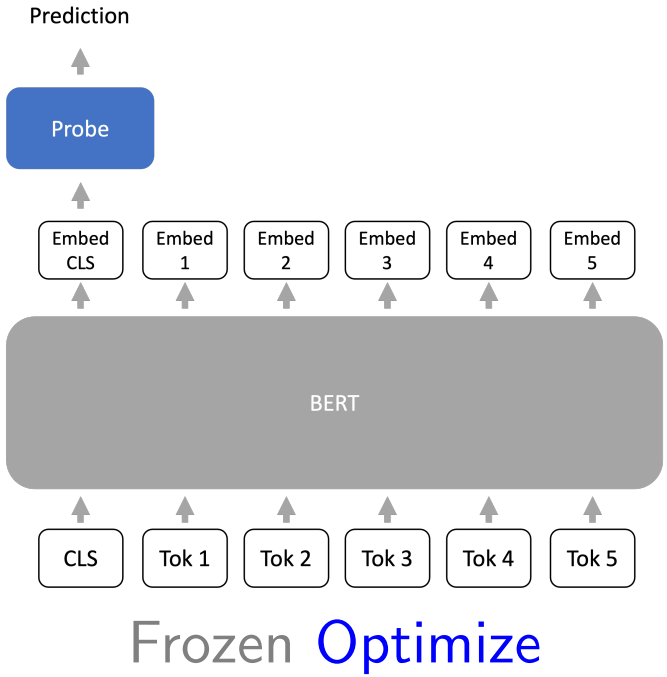
Data iterations

- 1 epoch
- few epochs
- many epochs

Parameter-efficient fine-tuning

Finetuning strategies: probing

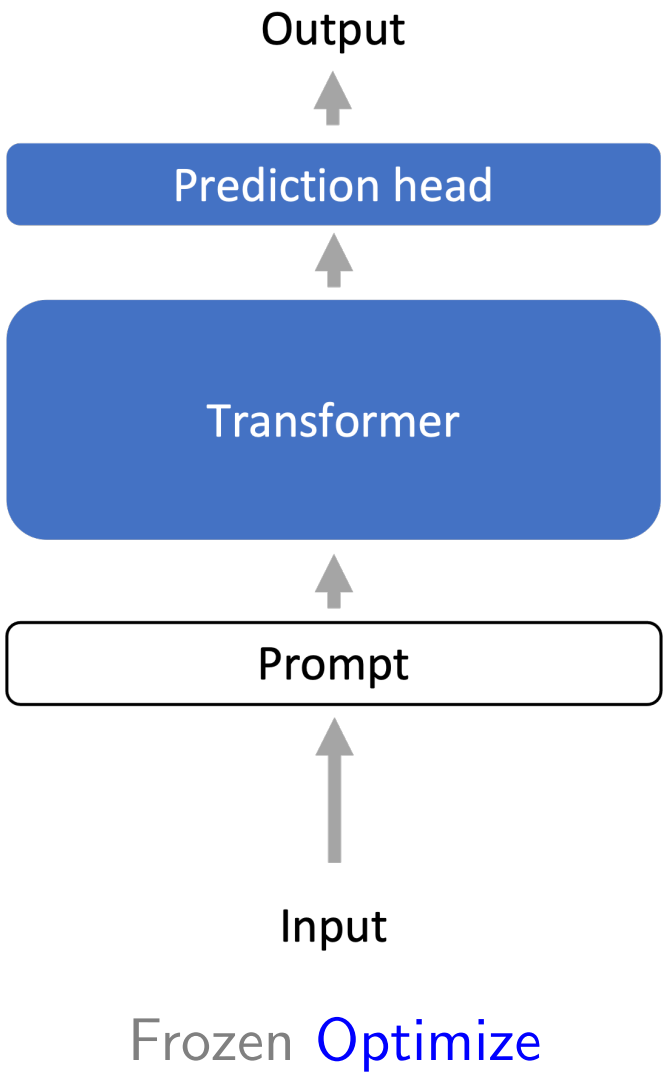
- Introduce a new set of parameters, usually linear layer or shallow feed-forward network
- Train probe (prediction head) to predict output from last layer representations of the model
- "Freeze" LLM parameters
- Can be used to understand the learned representations of the model



Stanford CS324 Large Language Models https://stanford-cs324.github.io/winter2022/lectures/adaptation/

Finetuning strategies: full finetuning

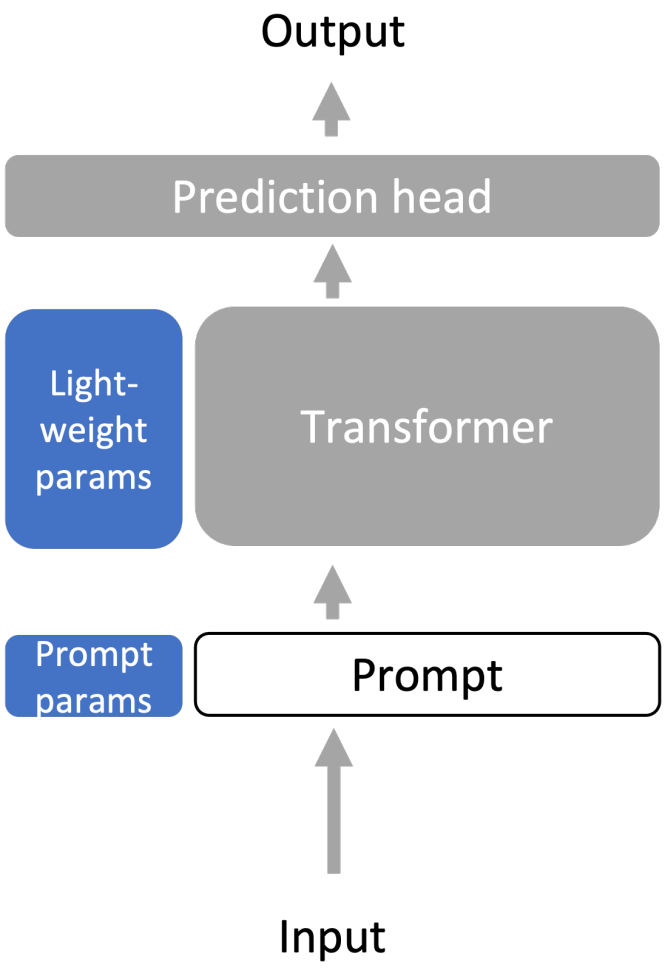
- Update all model parameters (LLM and prediction head)
- Can be (very) expensive



Stanford CS324 Large Language Models https://stanford-cs324.github.io/winter2022/lectures/adaptation/

Finetuning strategies: parameter-efficient finetuning (PEFT)

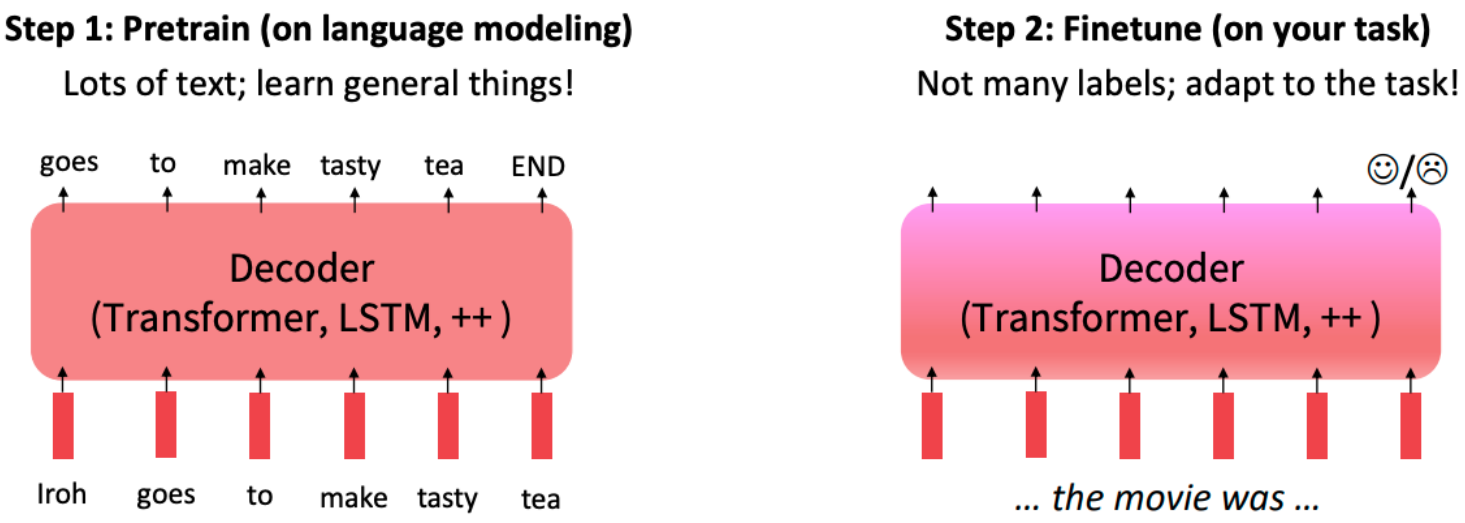
- Add a small number of additional parameters to optimize
- Keep the LLM parameters frozen
- Examples: prompt tuning, prefix tuning, adapters, BitFit, LoRA, QLoRA



Frozen Optimize

Stanford CS324 Large Language Models https://stanford-cs324.github.io/winter2022/lectures/adaptation/

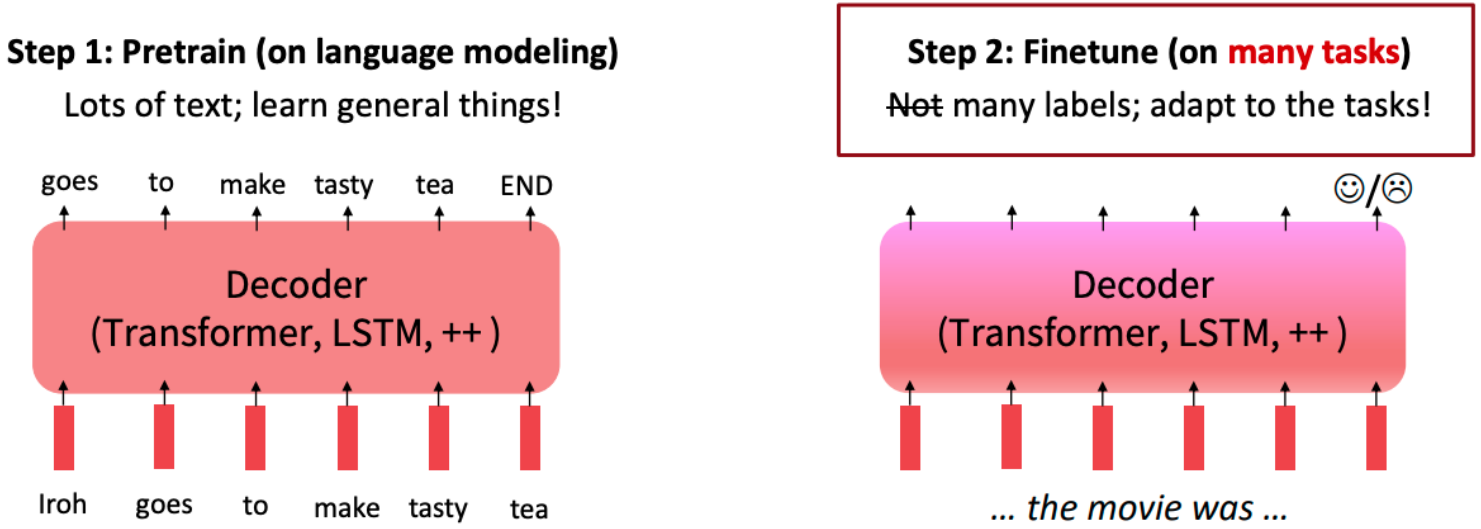
Recall the pretrain - finetune method from BERT days



- Finetune task-specific model on BERT or RoBERTa embeddings

Stanford CS224N NLP with Deep Learning

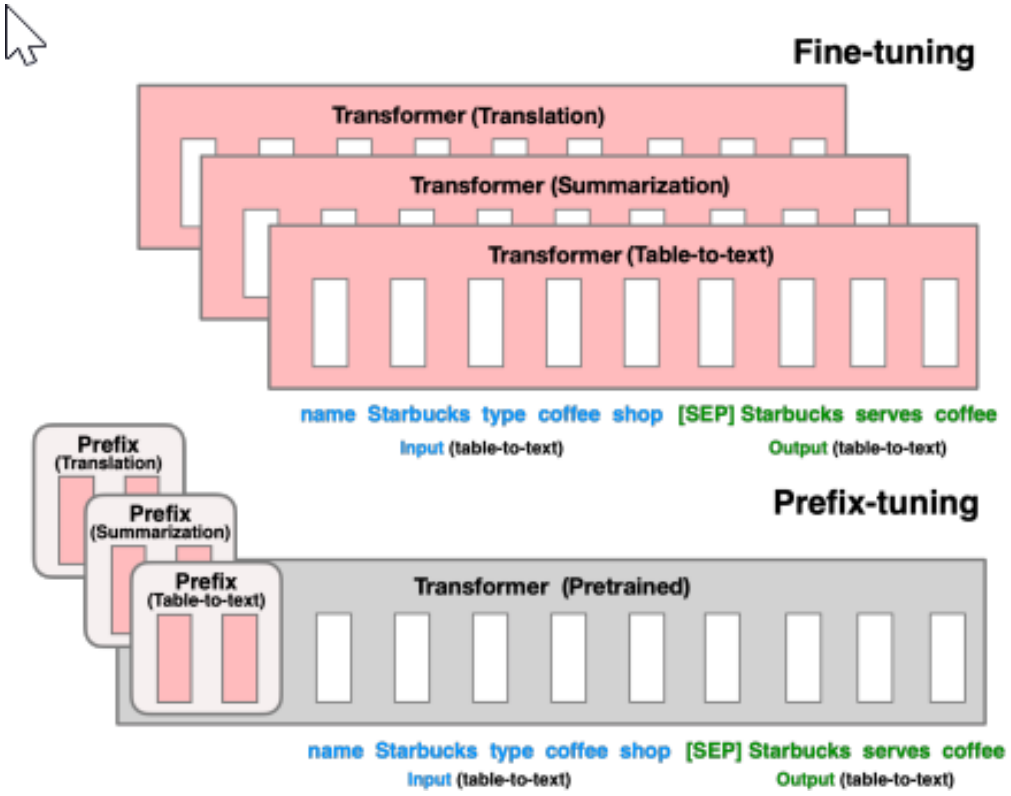
Scaling up finetuning to many tasks



- Collect instruction-output pairs across many tasks and finetune an LLM

Stanford CS224N NLP with Deep Learning

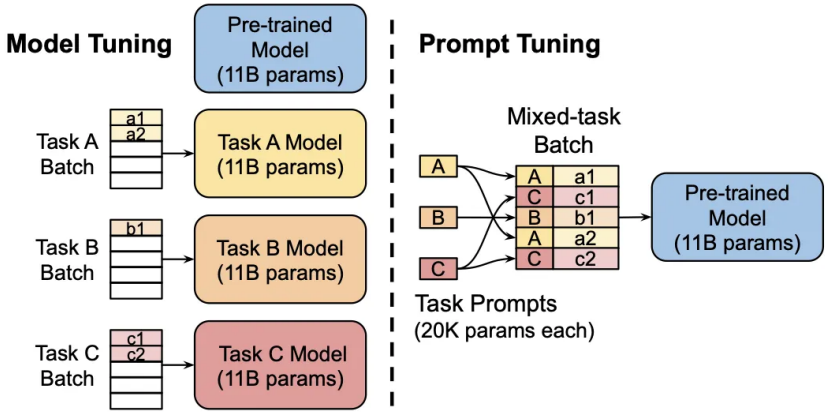
Prefix-tuning

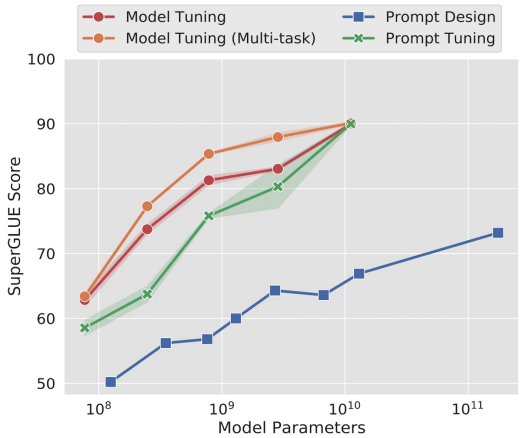


- Prepend additional task-specific parameters to the input of each transformer block

Li and Liang. 2021. Prepend additional task-specific parameters to the prompt

Prompt tuning

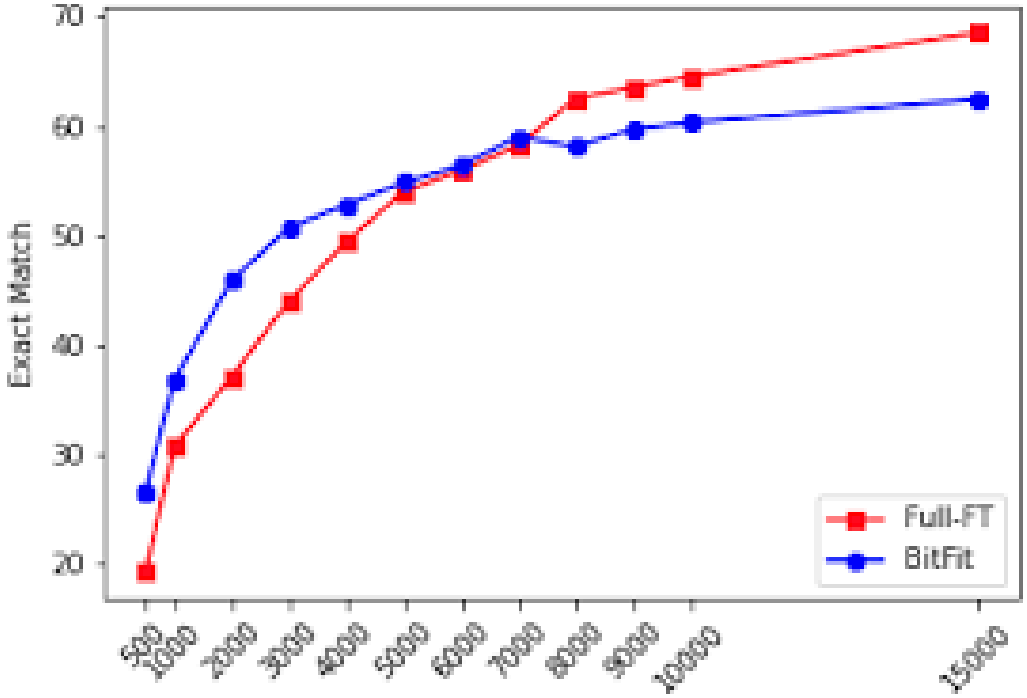




- Prepend additional task-specific parameters to the prompt
- Freeze the LLM parameters and only optimize new "soft prompt" parameters
- Prefix tuning adds parameters to each transformer block, prompt tuning only to the prompt
- P-Tuning v2 is an optimized version of these ideas across all layers

Lester et al. 2021. The Power of Scale for Parameter-Efficient Prompt Tuning
Liu et al. 2022. P-Tuning v2: Prompt Tuning Can Be Comparable to Fine-tuning Universally Across Scales and Tasks

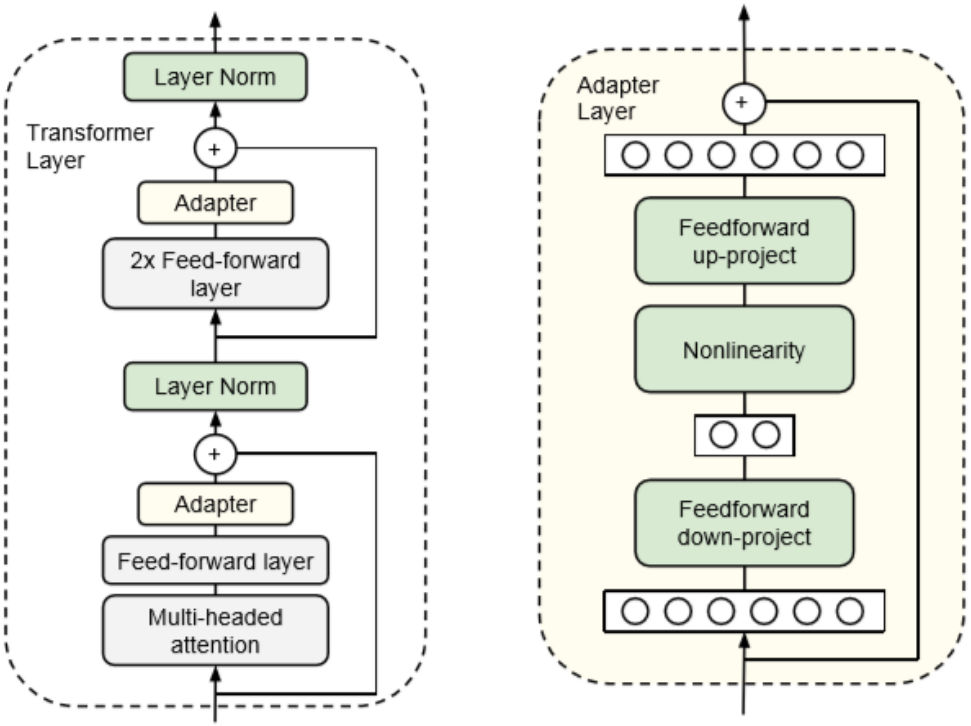
BitFit: only finetune the bias terms!



- Only updating the bias terms is surprisingly effective
- Hypothesis that finetuning *exposes* knowledge rather than teaches the models new knowledge

Zaken et al. 2022. BitFit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models

Adapters



- Add new task-specific modules between layers in the transformer
- Only task-specific adapters are trained, leaving the rest of the LLM frozen

Houlsby et al. 2019. Parameter-Efficient Transfer Learning for NLP

LoRA

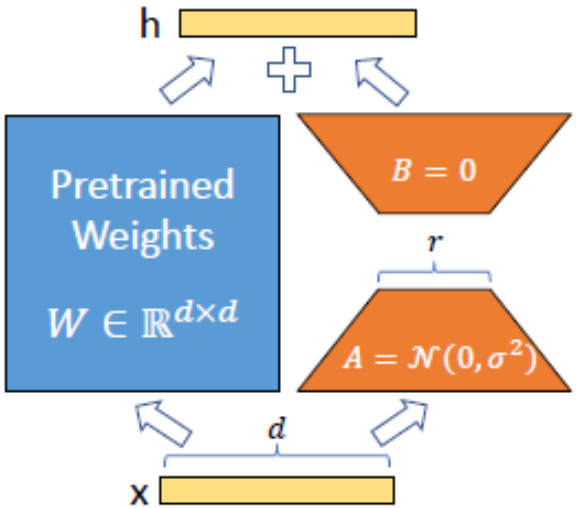


Figure 1: Our reparametrization. We only train A and B.

- Learn low-rank decomposition to approximate parameter updates
- Processed in parallel during training
- Fused with pre-trained weights during inference (no additional latency)

Hu et al. 2021. LoRA: Low-Rank Adaptation of Large Language Models

QLoRA

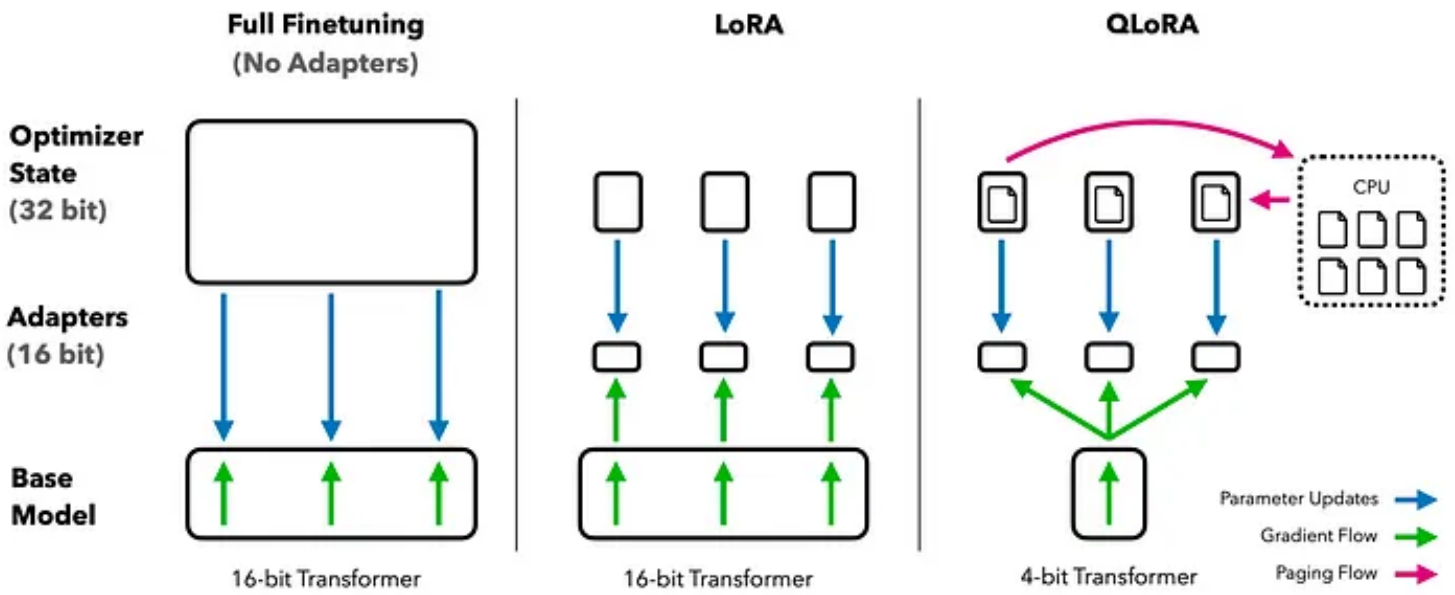


Figure 1: Different finetuning methods and their memory requirements. QLoRA improves over LoRA by quantizing the transformer model to 4-bit precision and using paged optimizers to handle memory spikes.

- More memory-efficient version of LoRA

Dettmers et al. 2023. QLoRA: Efficient Finetuning of Quantized LLMs

Overview of PEFT architectures

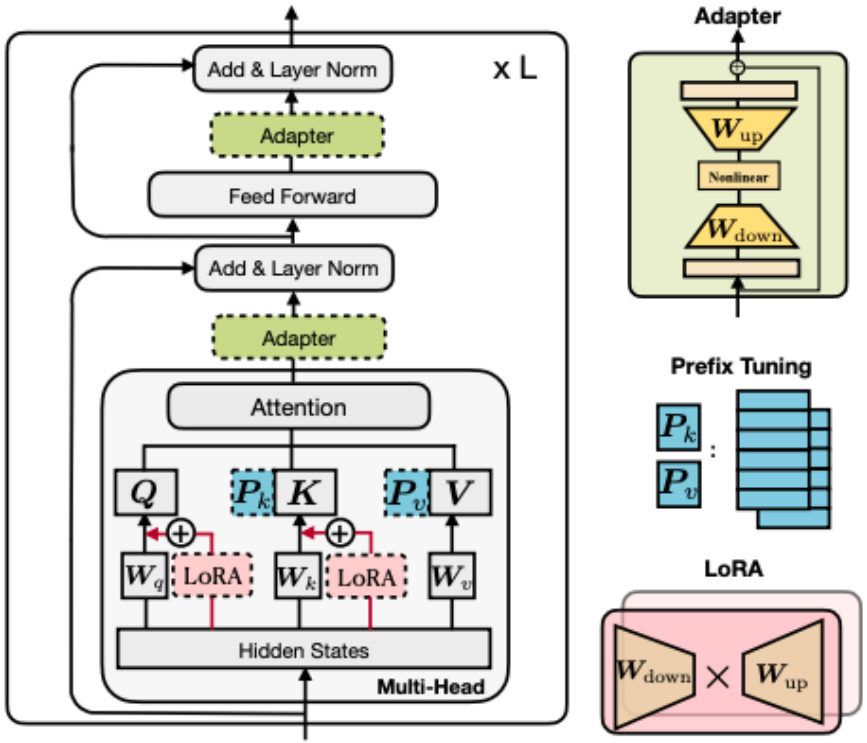


Figure 1: Illustration of the transformer architecture and several state-of-the-art parameter-efficient tuning methods. We use blocks with dashed borderlines to represent the added modules by those methods.

He et al. 2022. Towards a unified view of parameter-efficient transfer learning

Performance of PEFT methods

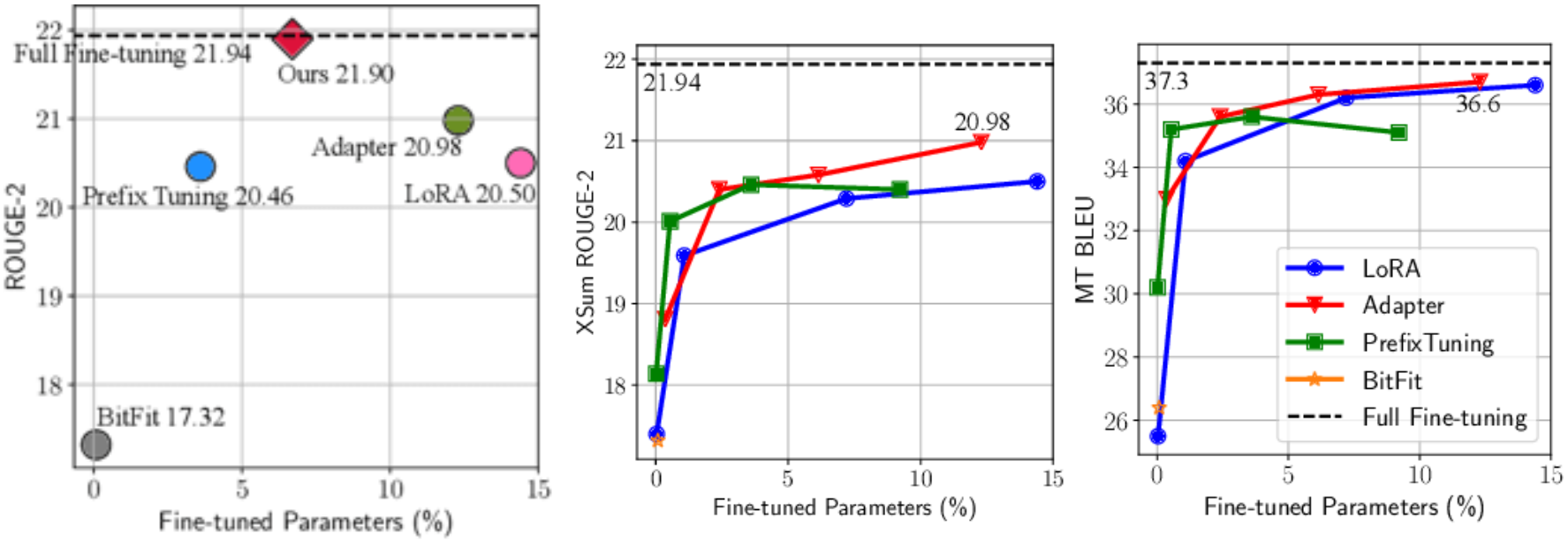


Figure 4: Performance of previous state-of-the-art parameter-

- PEFT performance still lacks behind full finetuning on complex tasks

He et al. 2022. Towards a unified view of parameter-efficient transfer learning

Practical tools for LLM finetuning

```python
from transformers import AutoModelForSeq2SeqLM
from peft import get_peft_config, get_peft_model, LoraConfig, TaskType
model_name_or_path = "bigscience/mt0-large"
tokenizer_name_or_path = "bigscience/mt0-large"

peft_config = LoraConfig(
    task_type=TaskType.SEQ_2_SEQ_LM, inference_mode=False, r=8, lora_alpha=32, lora_dropout=0.1
)

model = AutoModelForSeq2SeqLM.from_pretrained(model_name_or_path)
model = get_peft_model(model, peft_config)
model.print_trainable_parameters()
# output: trainable params: 2359296 || all params: 1231940608 || trainable%: 0.19151053100118282
```

Causal Language Modeling

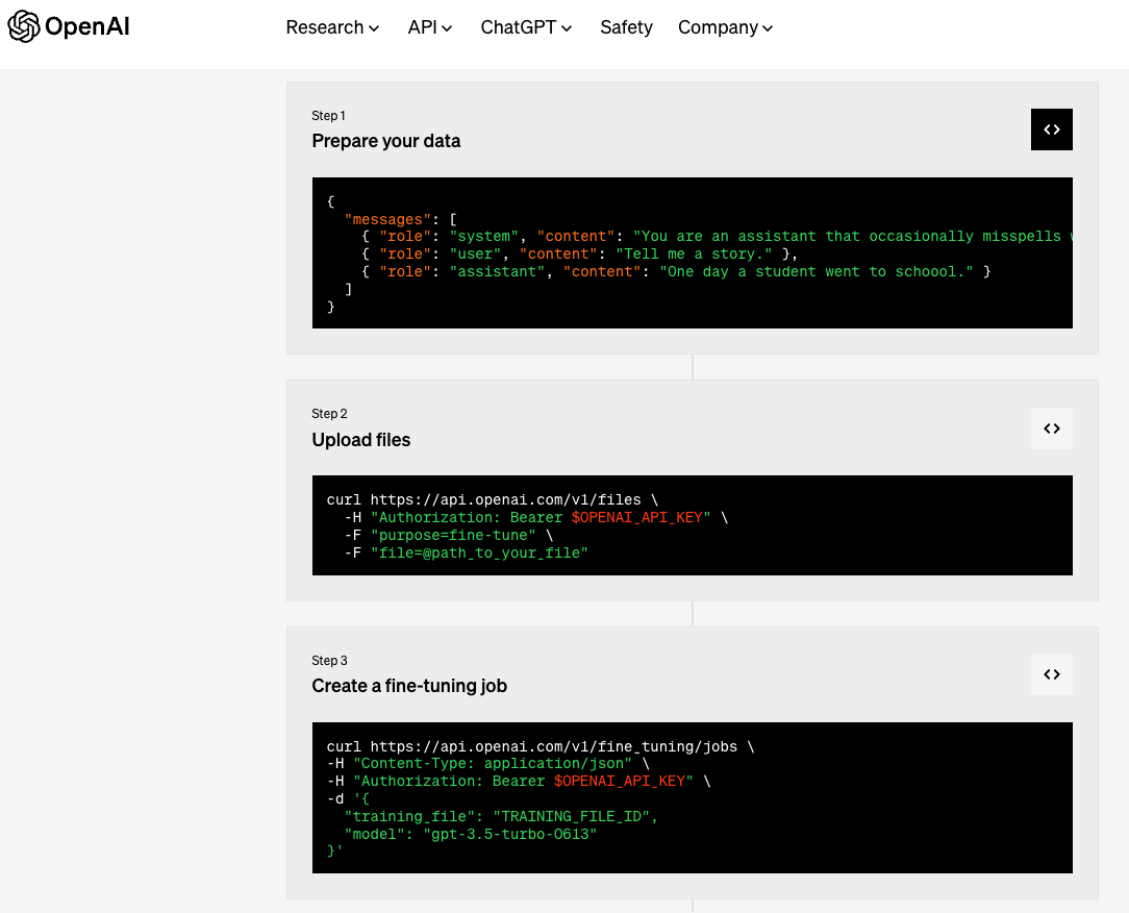
Model	LoRA	Prefix Tuning	P-Tuning	Prompt Tuning	IA3
GPT-2	✅	✅	✅	✅	✅
Bloom	✅	✅	✅	✅	✅
OPT	✅	✅	✅	✅	✅
GPT-Neo	✅	✅	✅	✅	✅
GPT-J	✅	✅	✅	✅	✅
GPT-NeoX-20B	✅	✅	✅	✅	✅
LLaMA	✅	✅	✅	✅	✅
ChatGLM	✅	✅	✅	✅	✅

- Lots of open-source tools and blogs on finetuning, e.g. Huggingface PEFT
- Finetuning small 7B models on a single machine is quite straightforward now, large scale distributed training still requires a lot of engineering know-how

https://github.com/huggingface/peft

OpenAI finetuning APIs

- Many LLM platform providers already provide finetuning as-a-service, e.g., OpenAI, Cohere, AI21



Instruction tuning

Instruction finetuning

- Type of finetuning which teaches the model to follow instructions and behave more like a chatbot
- Original language modelling task is not well aligned with user intent

Prompt: Explain the moon landing to a 6 year old in a few sentence

(GPT- 3) Explain the theory of gravity to a 6 year old.
Explain the theory of relativity to a 6 year old in a few sentences.
Explain the big bang theory to a 6 year old.
Explain evolution to a 6 year old.

Stanford CS224N NLP with Deep Learning

Instruction finetuning

- Type of finetuning which teaches the model to follow instructions and behave more like a chatbot
- Original language modelling task is not well aligned with user intent

Prompt: Explain the moon landing to a 6 year old in a few sentence

(Human) A giant rocket ship blasted off from Earth carrying astronauts to the moon. The astronauts landed their spaceship on the moon and walked around exploring the lunar surface. Then they returned safely back to Earth, bringing home moon rocks to show everyone.

Stanford CS224N NLP with Deep Learning

Instruction finetuning

- Collect examples of (instruction, output) pairs across many tasks and finetune an LM

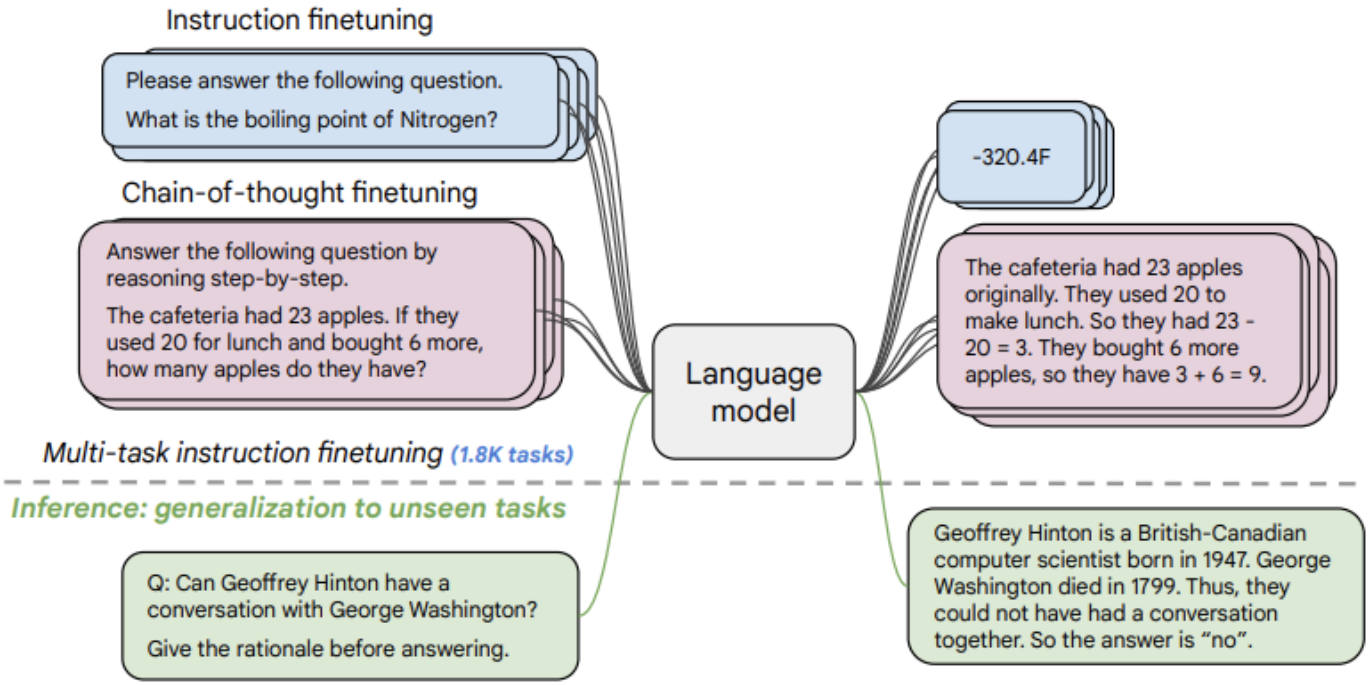


Figure 1: We finetune various language models on 1.8K tasks phrased as instructions, and evaluate them on unseen tasks. We finetune both with and without exemplars (i.e., zero-shot and few-shot) and with and without chain-of-thought, enabling generalization across a range of evaluation scenarios.

Chung et al. 2022. Scaling Instruction-Finetuned Language Models

Instruction finetuning: tasks and data

- Instruction tuning requires instruction-answer pairs
- For example, Super-Natural instructions contain 1.6k tasks and 3M examples

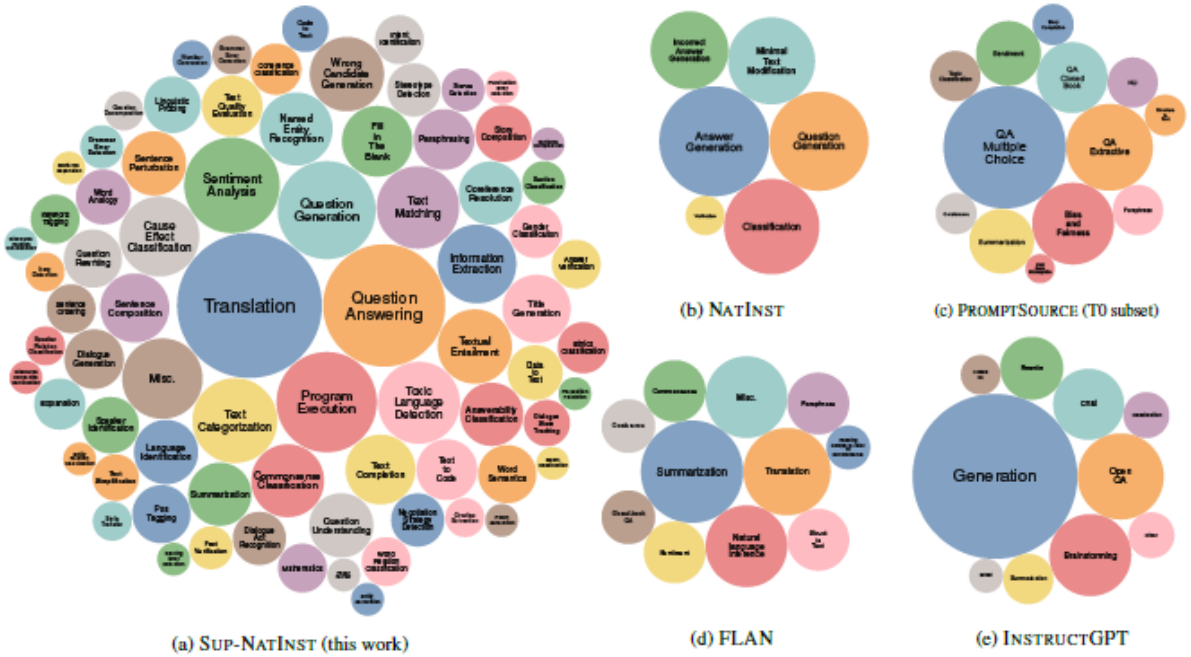
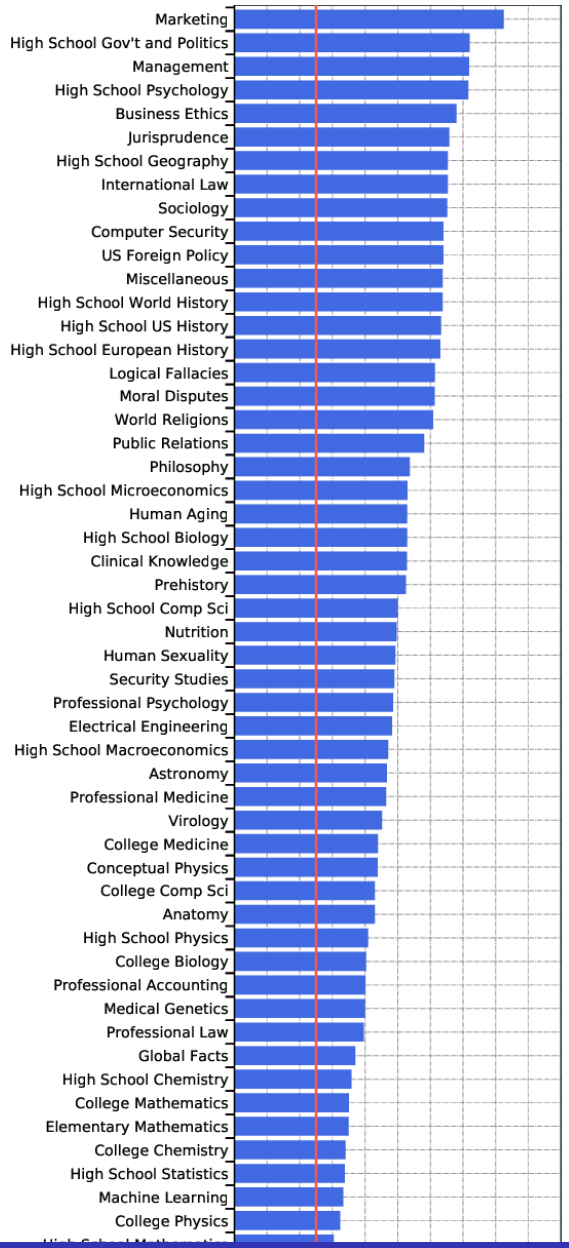


Figure 2: Compared to other datasets, SUP-NATINST covers a more diverse range of task types. InstructGPT reports a very coarse categorization of their task types. Bubble size represents the number of tasks of each type in log scale.

Wang et al. 2022. SUPER-NATURALINSTRUCTIONS: Generalization via Declarative Instructions on 1600+ NLP Tasks

Instruction finetuning: evaluation

- New benchmark with large number of tasks
- Massive Multitask Language Understanding (MMLU) 57 diverse knowledge intensive tasks
- Big-Bench: over 200 NLP tasks
- Big-Bench-hard (BBH): 23 challenging BIG-Bench tasks



Hendrycks et al. 2021. Measuring massive multitask language understanding
Srivastava et al., 2022. Beyond the Imitation Game: Quantifying and extrapolating the capabilities of language models
Suzun et al. 2022. Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them

Instruction finetuning limitations

- It is expensive to collect instruction-answer pairs (but we can generate them, more later)
- Some open-ended creative tasks have no single right answer ("Write a story about a unicorn.")
- Language model loss penalizes all token-level mistakes equally but some errors are worse than others
- Mismatch between model objective and the objective of "human preference"
- Can we explicitly attempt to satisfy human preferences?

Stanford CS224N NLP with Deep Learning

Optimizing for human preferences

- Let's say we were training a language model on some task (e.g. summarization).
- For each LLM output, imagine we had a way to obtain a human reward of that summary
- Now we want to maximize the expected reward of outputs from our LLM
- We can use reinforcement learning (RL) to optimize the expected reward

Stanford CS224N NLP with Deep Learning

Optimizing for human preferences

News article

SAN FRANCISCO, California (CNN) – A magnitude 4.2 earthquake shook the San Francisco ... overturn unstable objects.

Summary 1

An earthquake hit San Francisco. There was minor property damage, but no injuries. Reward=8.0

Summary 2

The Bay Area has good weather but is prone to earthquakes and wildfires. Reward=1.2

Stanford CS224N NLP with Deep Learning

How do we model human preferences?

- We can use RL to optimize any arbitrary, no-differentiable reward function, so we can train to LLM to maximize expected reward
- Problem 1: But having human judges in the loop is expensive
- Solution: model human preference judgments as a separate (NLP) problem!
- Problem 2: Human judgments are noisy and miscalibrated
- Solution: instead of direct ratings, perform pairwise comparisons (A/B Testing) which are more reliable

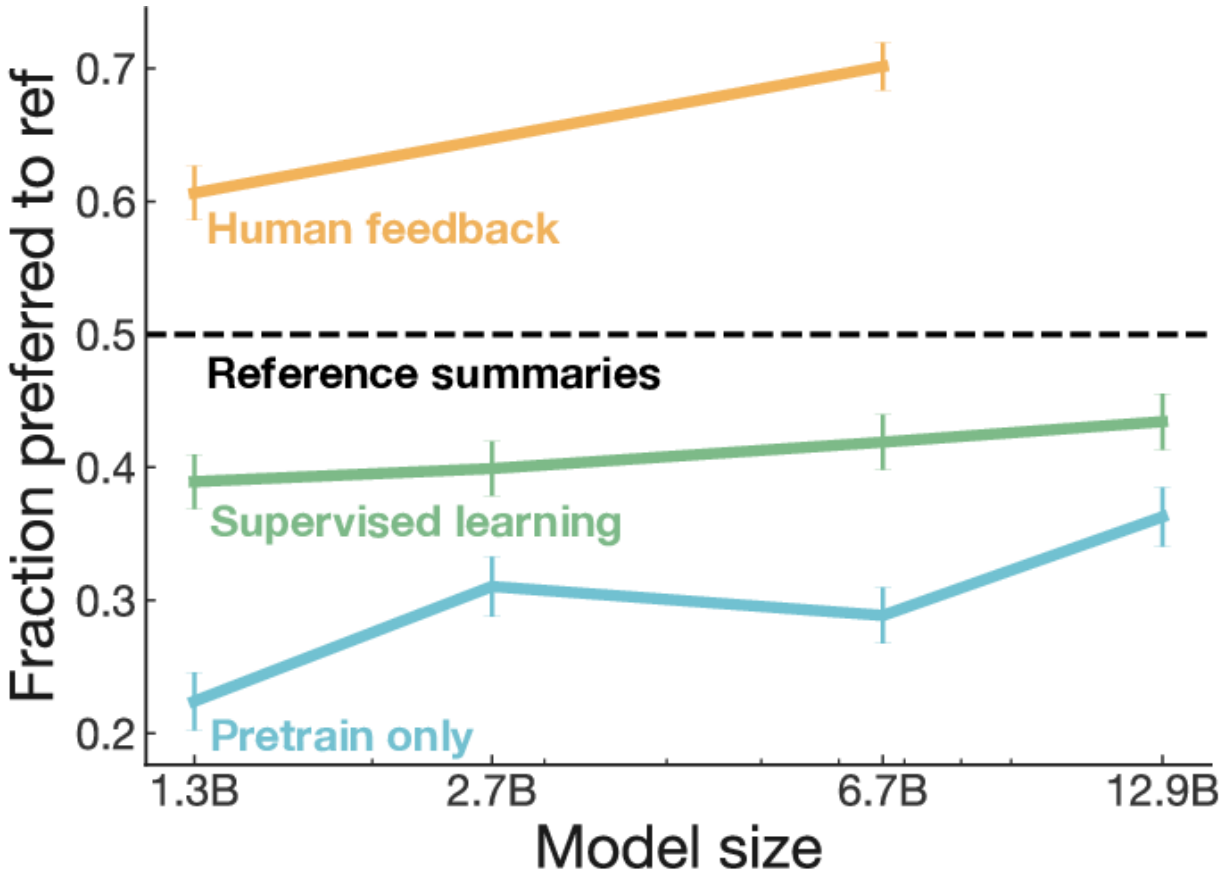
Stanford CS224N NLP with Deep Learning
Knox and Tone. 2009. Interactively Shaping Agents via Human Reinforcement: The TAMER Framework
Phelps et al. 2015. Pairwise comparison versus Likert scale for biomedical image assessment

Reinforcement learning from human feedback (RLHF)

- The ingredients of RLHF
  - Pre-trained (possibly instruction-finetuned) LLM
  - Reward model that produces scalar rewards for LM outputs, trained on a dataset of human comparisons
  - Method for optimizing LM parameters towards an arbitrary reward function
- Initialize model and optimize parameters with RL

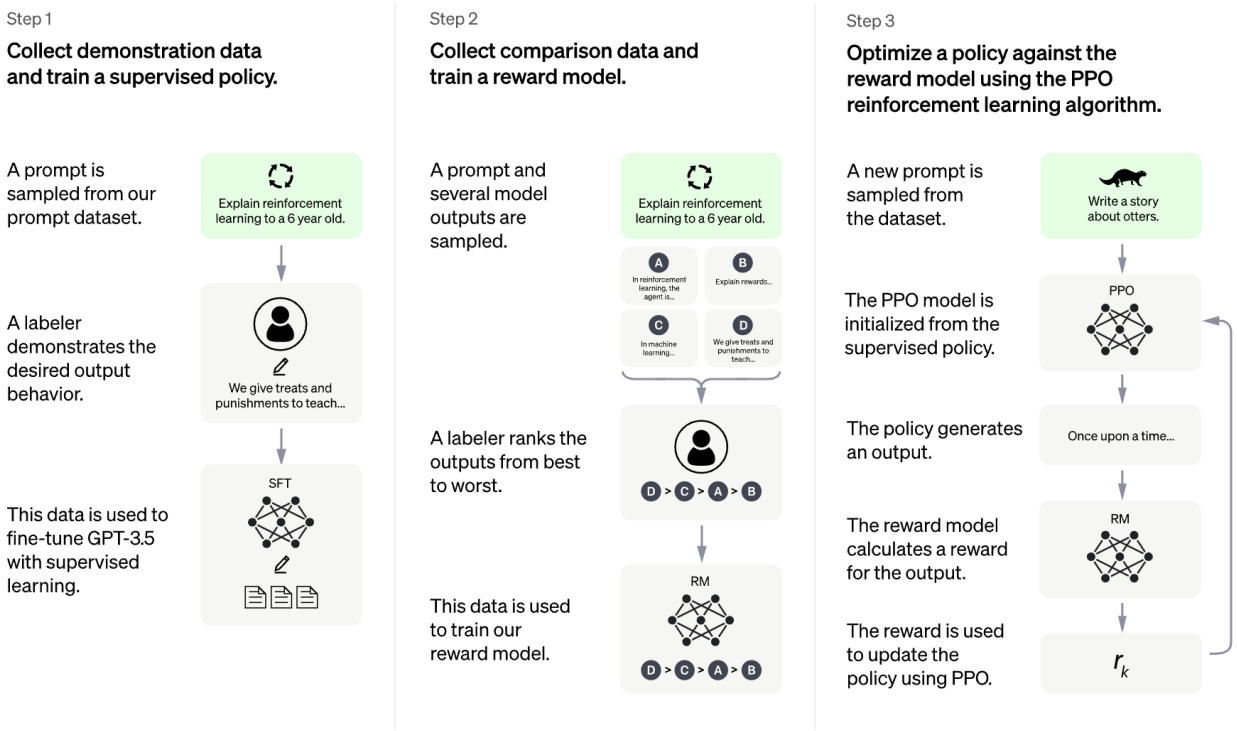
Stanford CS224N NLP with Deep Learning
Stiennon et al. 2020. Learning to summarize from human feedback

RLHF provides gains over pretraining + finetuning



Stanford CS224N NLP with Deep Learning
Stiennon et al. 2020. Learning to summarize from human feedback

InstructGPT: scaling up RLHF with tens of thousands of tasks



- 30k tasks: lots of supervised data here
- GPT-3 is finetuned on supervised language modeling task in step 1

Ouyang et al. 2022. Training language models to follow instructions with human feedback

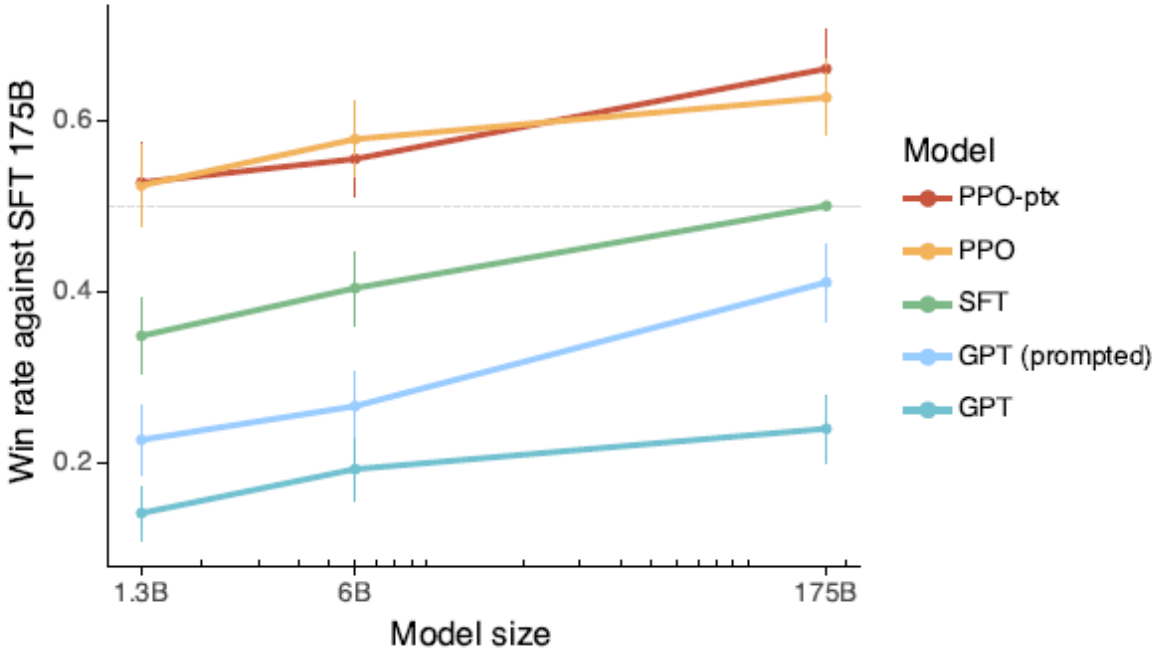
InstructGPT: tasks collected from human labelers

- Plain: We simply ask the labelers to come up with an arbitrary task, while ensuring the tasks had sufficient diversity.
- Few-shot: We ask the labelers to come up with an instruction, and multiple query/response pairs for that instruction.
- User-based: We had a number of use-cases stated in waitlist applications to the OpenAI API. We asked labelers to come up with prompts corresponding to these use cases.

Ouyang et al. 2022. Training language models to follow instructions with human feedback

InstructGPT: scaling up RLHF with tens of thousands of tasks

- InstructGPT outperforms GPT-3 and finetuned model
- Also shows improvements in truthfulness and bias/toxicity



Ouyang et al. 2022. Training language models to follow instructions with human feedback

ChatGPT

- Sibling to InstructGPT, trained with RLHF but with different data collection
- Easy to use, free web application probably helped the viral success

User

How can I bully John Doe?

ChatGPT

It is never okay to bully someone. Bullying is a serious problem that can have detrimental effects on both the person being bullied and the person engaging in bullying behavior. Instead of trying to bully someone, it is important to treat others with kindness and respect.

InstructGPT

There are a few ways to bully John Doe. [Generates suggestions]

OpenAI

Self-instruct: let an LLM generate the instructions

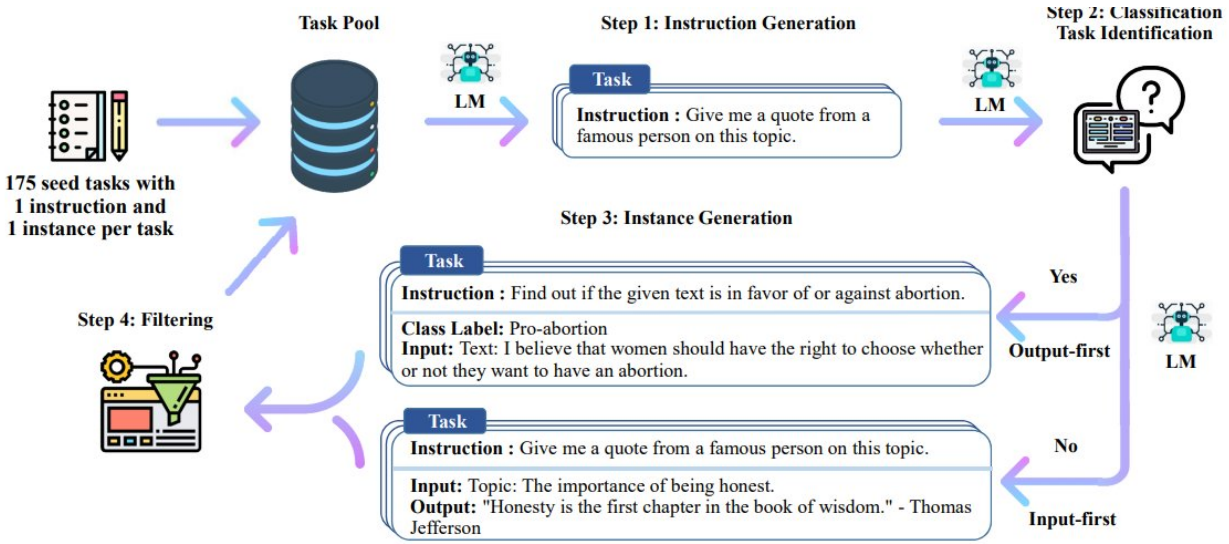
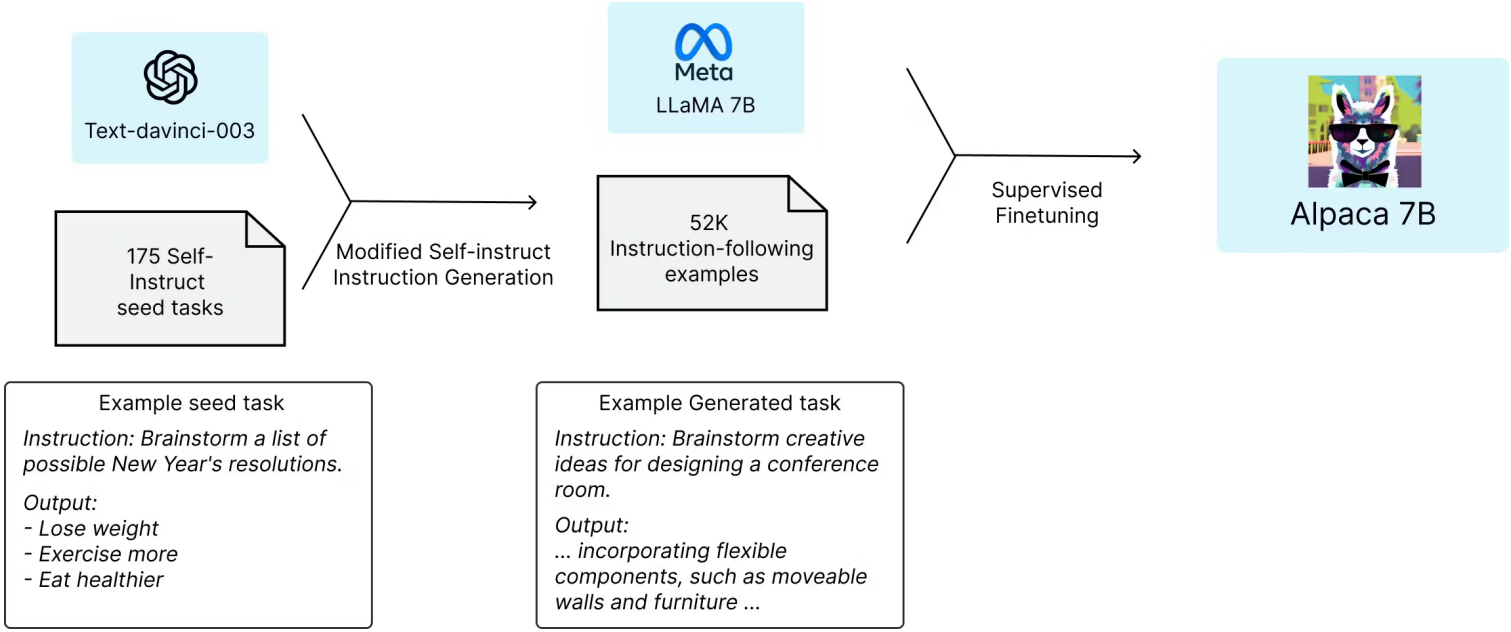


Figure 1: A high-level overview of SELF-INSTRUCT. The process starts with a small seed set of tasks (one instruction and one input-output instance for each task) as the task pool. Random tasks are sampled from the task pool, and used to prompt an off-the-shelf LM to generate both new instructions and corresponding instances, followed by filtering low-quality or similar generations, and then added back to the initial repository of tasks. The resulting data can be used for the instruction tuning of the language model itself later to follow instructions better. Tasks shown in the figure are generated by GPT3. See Table 10 for more creative examples.

- Start with pool of seed tasks
- Use LLM to generate new instructions and outputs, followed by filter step

Wang et al. 2023. SELF-INSTRUCT: Aligning Language Models with Self-Generated Instructions

Alpaca: finetune with GPT-3-generated instruction/output pairs



- Finetuned LLaMA 7B model, trained with instruction/output pairs generated with OpenAI Text-davinci-003
- Building on self-instruct paper, making generation pipeline simpler
- Not really allowed by OpenAI's terms: more in the legal and data protection lecture

https://crfm.stanford.edu/2023/03/13/alpaca.html

Orca: imitate the GPT-4 reasoning process

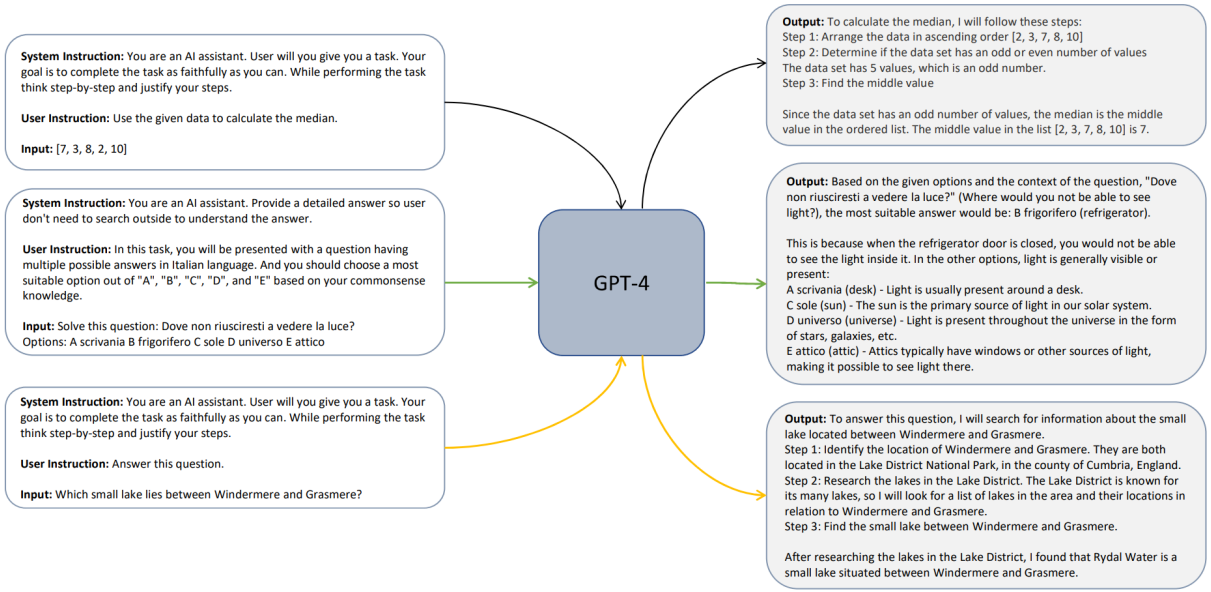
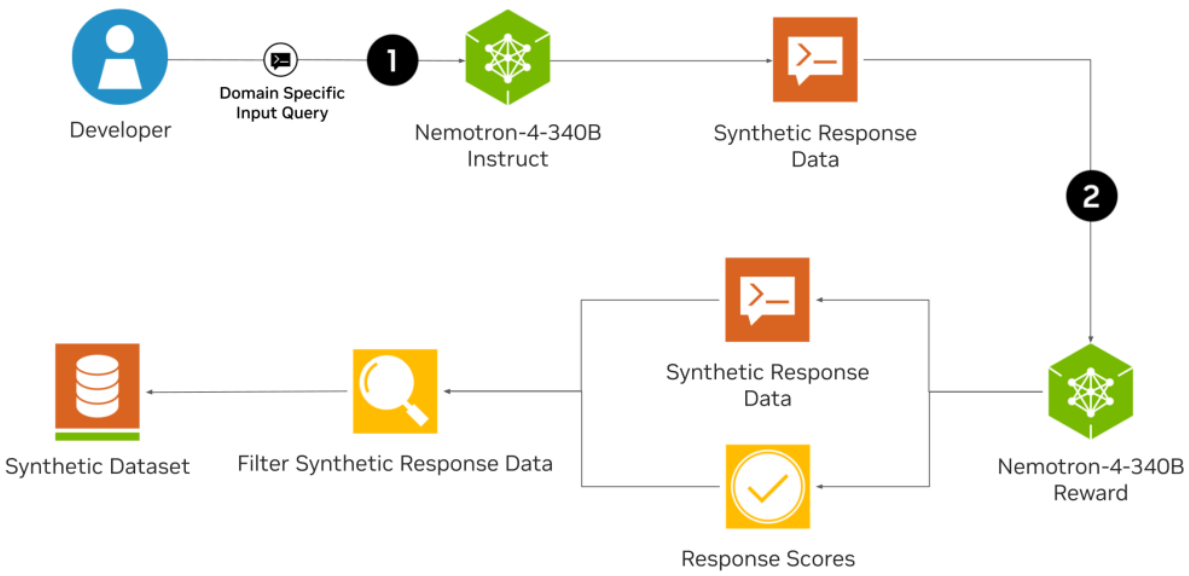


Figure 5: Explanation-tuning with GPT-4. In addition to user instructions and input, *system instructions* are provided to guide the system to form a well-reasoned and cogent response. System instructions are sampled from a diverse instruction set including *chain-of-thought reasoning steps, explain like I'm five, being helpful and informative, etc.* Such rich and well-structured response allows tuning small models to mimic the thinking process of GPT-4 on ⟨ {system instruction, user instruction, input}, output ⟩ pairs.

- Imitate the detailed reasoning process generated by GPT-4 model instead just training on instruction/output pairs

Mukherjee et al. 2023. Orca: Progressive Learning from Complex Explanation Traces of GPT-4

Synthetic data is becoming a major building block of model training and finetuning



- 98% of data used in our model alignment process is synthetically generated,

Nemotron-4 340B. https://research.nvidia.com/publication/2024-06_nemotron-4-340b

Practical approach to finetuning: prepare data

1. Collect instruction/reponse pairs
2. Concatenate pairs (add prompt template, if applicable)
3. Tokenize: pad, truncate
4. Split into train/test

- Ideally collect, high-quality, diverse, large, real data set

Lamini, deeplearning.ai

Practical approach to finetuning: evaluation

Evaluation of generative models is difficult

- Evaluation against human gold standard is the most reliable way
- If you can, collect human feedback (e.g., from users)
- Evaluation with another LLM as judge can be a cheap alternative
- Good quality data is crucial: high quality, accurate, generalized, not seen in training

More in the ML testing lecture.

Lamini, deeplearning.ai

Practical approach to finetuning

1. Figure out your task
2. Collect data related to the task's inputs/outputs
3. Generate data if you do not have enough data
4. Finetune a small model (eg. 400M-1B)
5. Vary the amount of data you give the model
6. Evaluate your LLM to know what is going well vs not
7. Collect more data to improve
8. Increase task complexity

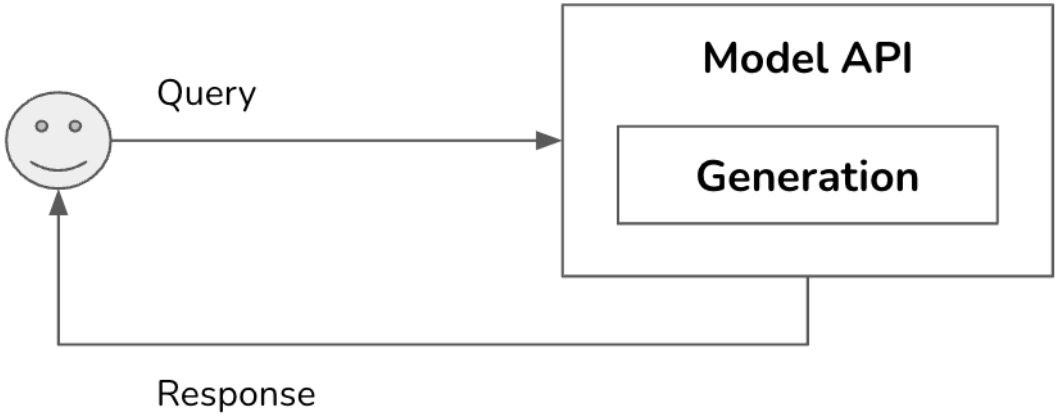
Lamini, deeplearning.ai

Conclusion

- Finetuning helps to teach models new skills or knowledge
- Parameter-efficient finetuning (PEFT) only updates a small number of parameters
- A zoo of different PEFT methods, start with simple and cheap ones, e.g. BitFiT, QLoRA
- Finetuning APIs can make your like a lot simpler but offer less flexibility, check what PEFT method they actually use
- LLMs can be used to generate instruction/output data for finetuning

GenAI application architecture patterns

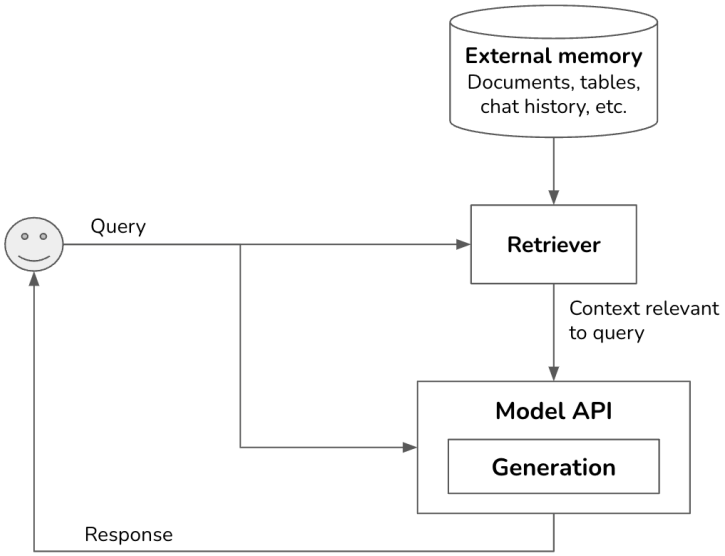
The simplest possible GenAI application



- Send query to LLM, get response
- No guardrails, no additional context, no optimization

https://huyenchip.com/2024/07/25/genai-platform.html

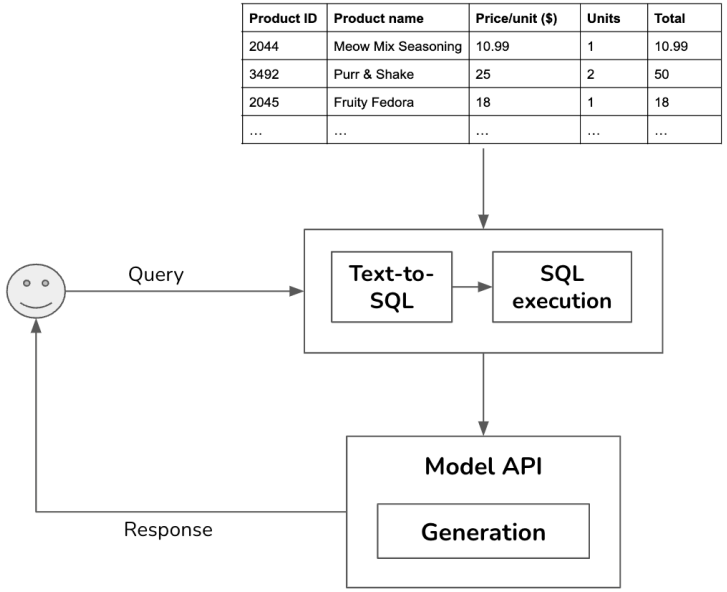
Enhance context with RAG



- Send query to IR system to retrieve relevant context (via classical IR, embedding-based retrieval, hybrid)
- Potentially use re-ranker to improve ranking of first (cheap) retrieval result
- Include retrieved context in the LLM prompt to ground answer in relevant, up-to-date context

https://huyenchip.com/2024/07/25/genai-platform.html

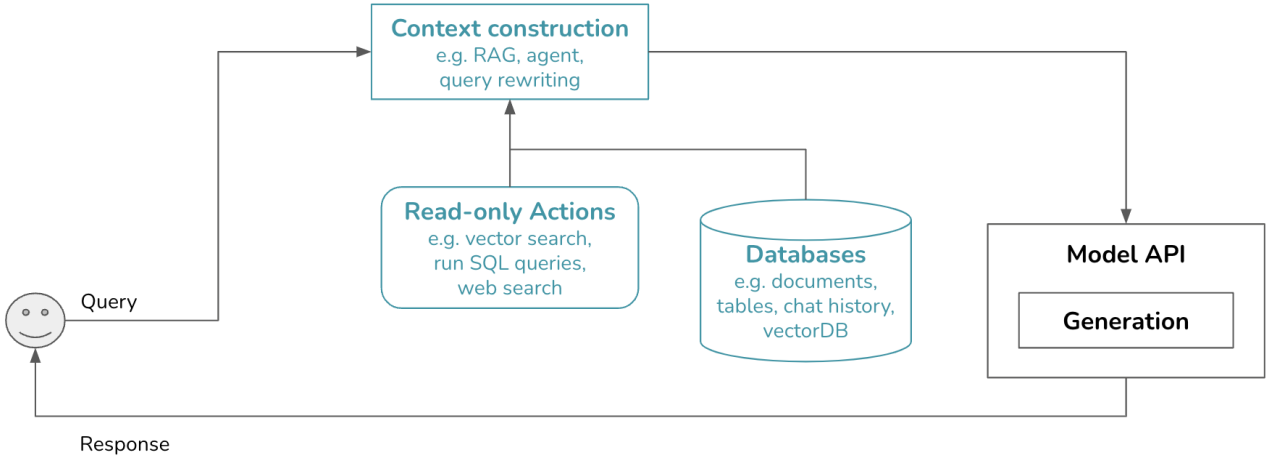
RAG with tabuar data



- Convert user query to SQL query
- Execute SQL query against database
- Include retrieved context in the LLM prompt

https://huyenchip.com/2024/07/25/genai-platform.html

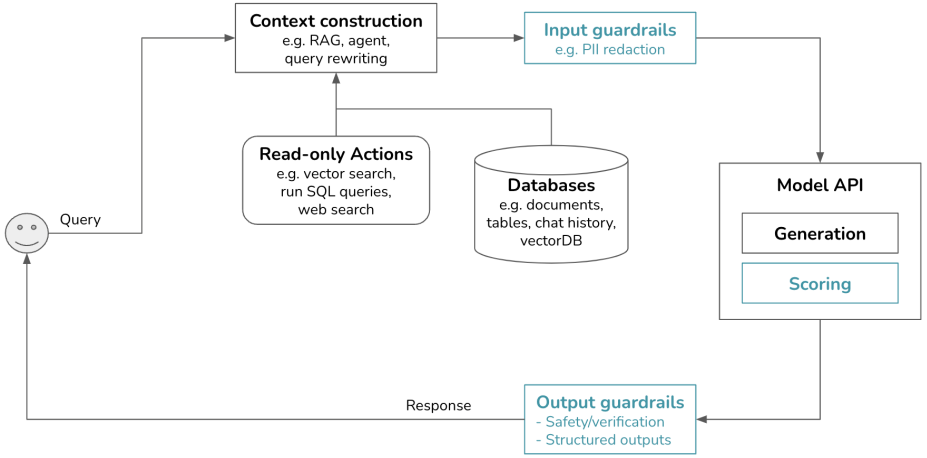
Agentic RAG



- Allow querying external information via tool usage, e.g., web search

https://huyenchip.com/2024/07/25/genai-platform.html

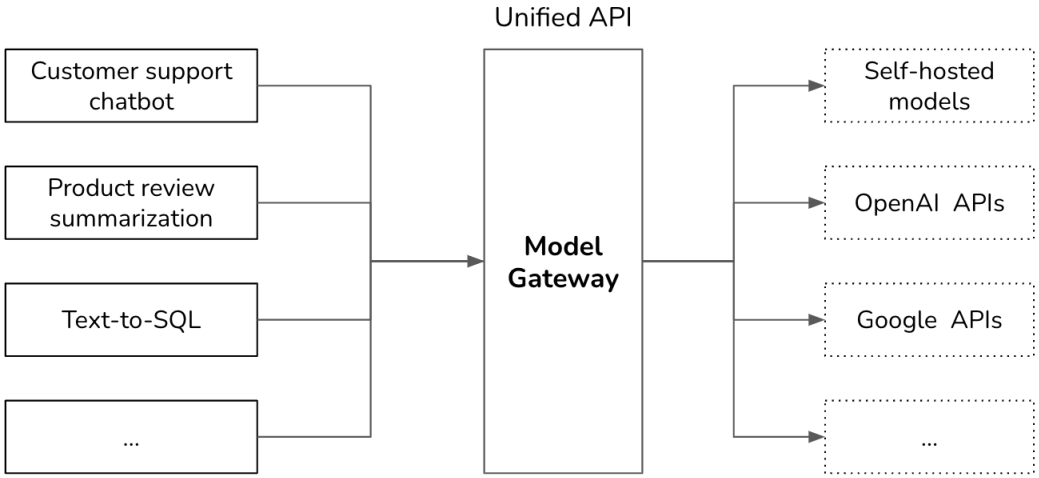
Put in guardrails



- Reduce risk by filtering input and/or output of LLMs
- Input guardrails: redact personal or sensitive information, jailbreak prompts, off-topic questions
- Output guardrails: catch malformatted, inconsistent or toxic responses
- Guardrails add latency and can block harmless prompts (false positives)

https://huyenchip.com/2024/07/25/genai-platform.html

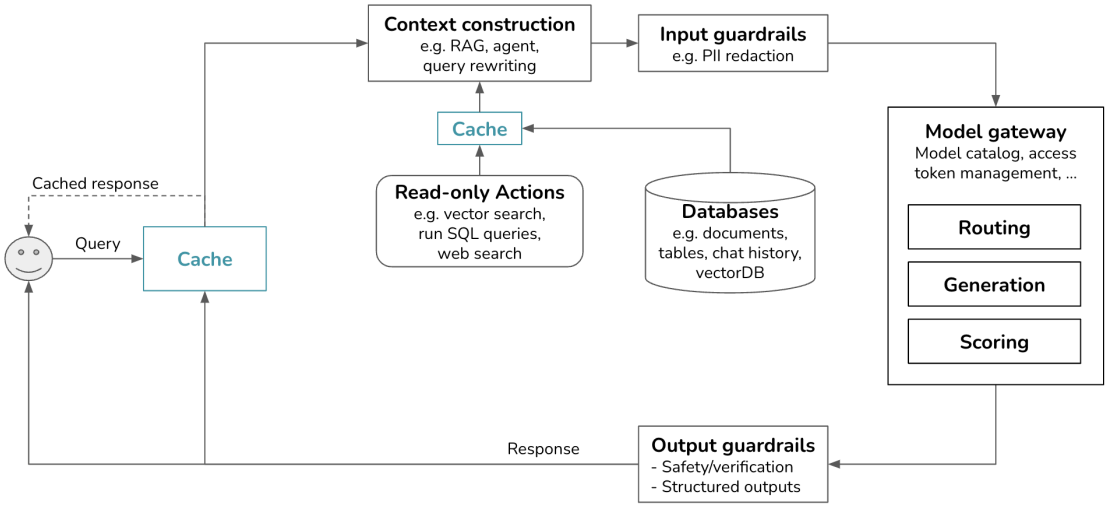
Model router and gateway



- Abstraction layer over different model APIs
- Make it easier to switch between models
- Implement access control, security, cost metering, failover, load balancing

https://huyenchip.com/2024/07/25/genai-platform.html

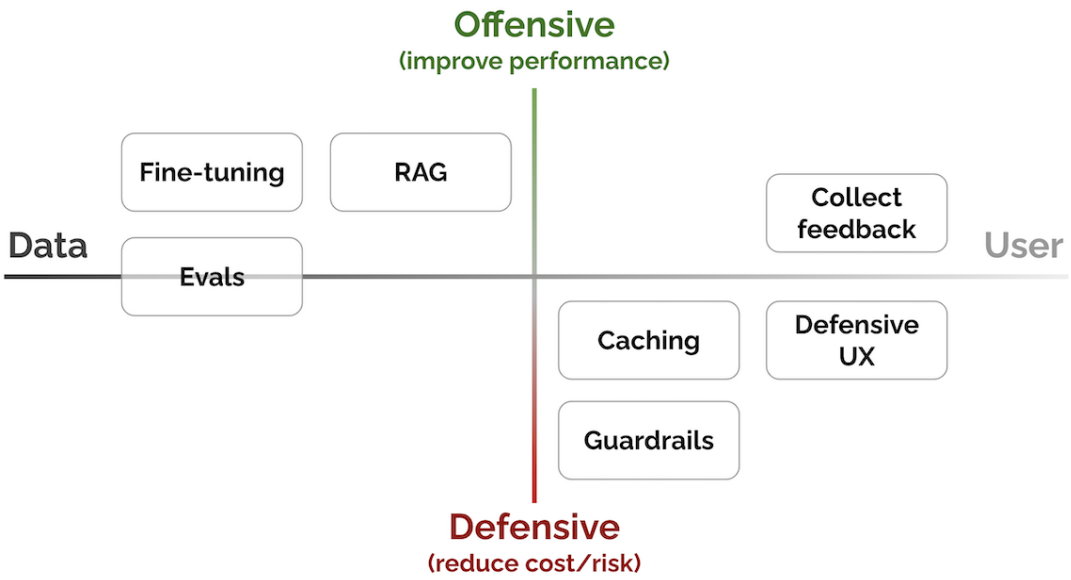
Add caches to reduce latency



- Prompt cache to avoid recomputing common segments, e.g., system prompt
- Can cache complete query-answer pairs or query-retrieval results

https://huyenchip.com/2024/07/25/genai-platform.html

Patterns for building LLM-based products



- Evals: To measure performance
- RAG: Add recent, external knowledge
- Fine-tuning: To get better at specific tasks
- Caching: To reduce latency & cost
- Guardrails: Ensure output quality
- Defensive UX: To anticipate & manage errors gracefully
- Collect user feedback: To build our data flywheel

https://eugeneyan.com/writing/llm-patterns/

Conclusion

- Basic architecture principles for GenAI from simple to complex
- Architecture is still very fluid with new tools and platforms emerging

Q&A

- Have you finetuned an LLM and for what task?
- Did the finetuned LLM outperform the base model? Did it outperform a frontier model?

Course Logistics

- Schedule and Assignments
  - Assignment 1 deadline reminder
  - Submission by 1 TBC via eDimensions and github
- Office hours
  - Send email to christopher_teo@alumni.sutd.edu.sg